

FleetMan

Plateforme de Gestion de Flotte Intelligente

Cahier d'Analyse et de Conception

Nouvelles Fonctionnalités — Vision Expert Métier

Type : **Cahier des Charges Fonctionnel & Technique**

Domaine : **Gestion de Flotte de Véhicules (B2B)**

Périmètre : **Backend Spring Boot — Nouvelles Fonctionnalités**

Modules : **8 nouveaux modules proposés**

Version : **1.0 — Mai 2026**

Niveau : **5^{ème} année Génie Informatique**

Table des matières

Introduction	2
1 Exigences Globales du Système Étendu	4
1.1 Exigences Fonctionnelles Transversales	4
1.2 Exigences Non-Fonctionnelles	5
1.3 Schéma d'Extension de la Base de Données	5
2 Module 1 — Planification et Ordonnancement	7
2.1 Exigences Fonctionnelles	7
2.2 Modèle de Données	8
2.3 Endpoints REST Proposés	9
3 Module 2 — Gestion des Contrats et Documents Légaux	10
3.1 Exigences Fonctionnelles	10
3.2 Modèle de Données	11
3.3 Endpoints REST Proposés	12
4 Module 3 — Tableau de Bord KPI et Rapports Avancés	14
4.1 KPIs Métier Identifiés	14
4.2 Exigences Fonctionnelles	15
4.3 Modèle de Données	15
4.4 Endpoints REST Proposés	16
5 Module 4 — Gestion des Dépenses et Budget	18
5.1 Exigences Fonctionnelles	18
5.2 Modèle de Données	19
5.3 Endpoints REST Proposés	20
6 Module 5 — Scoring Conducteur et Éco-conduite	21
6.1 Exigences Fonctionnelles	21
6.2 Algorithme de Scoring	22
6.3 Modèle de Données	22
6.4 Endpoints REST Proposés	23
7 Module 6 — Maintenance Préventive Intelligente	25

7.1	Exigences Fonctionnelles	25
7.2	Modèle de Données	26
7.3	Endpoints REST Proposés	27
8	Module 7 — Gestion des Clients et Missions	28
8.1	Exigences Fonctionnelles	28
8.2	Modèle de Données	29
8.3	Endpoints REST Proposés	31
9	Module 8 — Alertes et Règles Métier Configurables	33
9.1	Exigences Fonctionnelles	33
9.2	Modèle de Données	34
9.3	Endpoints REST Proposés	35
10	Plan d'Implémentation Détaillé	36
10.1	Principes Directeurs	36
10.2	Roadmap par Sprints	36
10.2.1	Sprint 1 — Fondations Transversales (Semaine 1-2)	37
10.2.2	Sprint 2 — Module Documents Légaux (Semaine 3-4)	37
10.2.3	Sprint 3 — Module KPI et Rapports (Semaine 5-6)	38
10.2.4	Sprint 4 — Module Dépenses et Budget (Semaine 7-8)	38
10.2.5	Sprint 5 — Module Planification (Semaine 9-11)	39
10.2.6	Sprint 6 — Module Scoring Conducteur (Semaine 12-14)	39
10.2.7	Sprint 7 — Module Maintenance Préventive (Semaine 15-16)	40
10.2.8	Sprint 8 — Module Clients et Missions (Semaine 17-20)	40
10.2.9	Sprint 9 — Module Alertes Configurables (Semaine 21)	41
10.3	Tableau Récapitulatif du Plan	41
10.4	Diagramme de Dépendances	42
11	Architecture Technique des Nouveaux Modules	43
11.1	Patron d'Implémentation Standard	43
11.2	Jobs Planifiés — Architecture	43
11.3	Nouvelles Migrations Liquibase	44
11.4	Nouveaux Tags Swagger	45
12	Analyse de Valeur et Priorisation	46
12.1	Matrice Valeur / Effort	46
12.2	Retour sur Investissement Estimé	46
	Conclusion	48

A	Récapitulatif des Nouvelles Tables SQL	49
B	Récapitulatif des Nouveaux Endpoints REST	50

Introduction

Contexte

Le backend FleetMan dispose aujourd'hui d'une base solide couvrant la gestion des véhicules, des conducteurs, des flottes, des trajets, du géorepérage et des opérations terrain (maintenances, incidents, carburant). Cependant, une plateforme de gestion de flotte professionnelle à destination des entreprises de transport africaines doit aller bien au-delà de ces fonctionnalités de base pour être compétitive face aux solutions internationales (Samsara, Verizon Connect, Geotab).

Objectif du Document

Ce cahier d'analyse et de conception identifie **8 nouveaux modules fonctionnels** à fort impact métier, issus de l'expertise du domaine de la gestion de flotte. Pour chaque module, il détaille :

- Les exigences fonctionnelles et non-fonctionnelles
- Le modèle de données (tables SQL supplémentaires)
- Les endpoints REST proposés
- La priorité d'implémentation et l'effort estimé

Synthèse des Modules Proposés

#	Module	Valeur Métier	Priorité	Effort
1	Planification & Ordonnancement	Optimisation des tournées et affectations	P1 — Critique	L
2	Gestion des Contrats & Documents	Conformité légale et alertes d'expiration	P1 — Critique	M
3	Tableau de Bord KPI Avancé	Pilotage de la performance opérationnelle	P1 — Critique	M
4	Gestion des Dépenses & Budget	Maîtrise des coûts d'exploitation	P2 — Haute	M

#	Module	Valeur Métier	Priorité	Effort
5	Scoring Conducteur	Sécurité routière et éco-conduite	P2 Haute	— L
6	Maintenance Préventive	Réduction des pannes et coûts	P2 Haute	— M
7	Gestion des Clients & Missions	Facturation et suivi des prestations	P3 Normale	— L
8	Alertes & Règles Métier	Automatisation de la supervision	P3 Normale	— S

TAB. 1 : Vue d'ensemble des 8 modules proposés
(S=Small, M=Medium, L=Large)

Note Experte

Les priorités sont définies selon l'impact métier immédiat pour une entreprise de transport en Afrique centrale. P1 = bloquant pour la commercialisation, P2 = différenciateur concurrentiel, P3 = enrichissement de l'offre.

Chapitre 1

Exigences Globales du Système Étendu

1.1 Exigences Fonctionnelles Transversales

Exigence

EF-T01 — Cloisonnement des données : Chaque gestionnaire de flotte ne doit accéder qu'aux données de ses propres flottes, véhicules et conducteurs. Aucune fuite de données inter-organisations n'est tolérée.

Exigence

EF-T02 — Traçabilité des actions : Toute création, modification ou suppression d'une ressource métier doit être horodatée et associée à l'utilisateur ayant effectué l'action (`created_by`, `updated_by`, `deleted_at`).

Exigence

EF-T03 — Soft Delete : Les suppressions de ressources critiques (véhicules, conducteurs, contrats) doivent être logiques (champ `deleted_at`) et non physiques, pour préserver l'historique.

Exigence

EF-T04 — Pagination et filtrage : Tous les endpoints de listing doivent supporter la pagination (`page`, `size`), le tri (`sort`) et le filtrage par critères métier.

Exigence

EF-T05 — Export de données : Les données tabulaires (rapports, historiques) doivent être exportables en CSV et PDF via des endpoints dédiés.

1.2 Exigences Non-Fonctionnelles

Code	Catégorie	Exigence
ENF-01	Performance	Temps de réponse $\leq 200\text{ms}$ pour 95% des requêtes en charge normale (100 utilisateurs simultanés)
ENF-02	Disponibilité	Disponibilité $\geq 99,5\%$ (hors maintenance planifiée)
ENF-03	Scalabilité	Architecture stateless permettant le scaling horizontal (Kubernetes)
ENF-04	Sécurité	Chiffrement TLS 1.3 en transit, données sensibles chiffrées au repos (AES-256)
ENF-05	Conformité	Respect du RGPD pour les données personnelles des conducteurs
ENF-06	Réactivité	Latence GPS ≤ 5 secondes entre position réelle et affichage
ENF-07	Résilience	Circuit Breaker sur tous les appels aux services externes
ENF-08	Observabilité	Logs structurés (JSON), métriques Prometheus, traces distribuées
ENF-09	Maintenabilité	Couverture de tests $\geq 70\%$ sur la couche domaine
ENF-10	Internationalisation	Support multilingue (FR, EN) pour les messages d'erreur et notifications

TAB. 1.1 : Exigences non-fonctionnelles du système étendu

1.3 Schéma d'Extension de la Base de Données

Les nouvelles tables s'intègrent dans le schéma `fleet` existant. Le tableau suivant liste les 18 nouvelles tables proposées :

Table	Module	Description
<code>fleet.schedules</code>	Planification	Plannings de service
<code>fleet.assignments</code>	Planification	Affectations véhicule-conducteur-mission

Table	Module	Description
<code>fleet.vehicle_documents</code>	Documents	Documents légaux des véhicules
<code>fleet.driver_documents</code>	Documents	Documents légaux des conducteurs
<code>fleet.document_alerts</code>	Documents	Alertes d'expiration
<code>fleet.kpi_snapshots</code>	KPI	Instantanés de KPIs périodiques
<code>fleet.expense_categories</code>	Dépenses	Catégories de dépenses
<code>fleet.expenses</code>	Dépenses	Dépenses opérationnelles
<code>fleet.budgets</code>	Dépenses	Budgets par flotte/période
<code>fleet.driver_scores</code>	Scoring	Scores de conduite
<code>fleet.driving_events</code>	Scoring	Événements de conduite (excès vitesse, etc.)
<code>fleet.maintenance_plans</code>	Maintenance	Plans de maintenance préventive
<code>fleet.maintenance_tasks</code>	Maintenance	Tâches de maintenance planifiées
<code>fleet.clients</code>	Clients	Clients de l'entreprise
<code>fleet.missions</code>	Clients	Missions/prestations
<code>fleet.invoices</code>	Clients	Factures
<code>fleet.alert_rules</code>	Alertes	Règles d'alerte configurables
<code>fleet.alert_history</code>	Alertes	Historique des alertes déclenchées

TAB. 1.2 : 18 nouvelles tables proposées

Chapitre 2

Module 1 — Planification et Ordonnancement

Résumé du Module

Priorité : P1 — Critique **Effort :** Large (3-4 semaines) **Impact :** Très élevé

Permet aux gestionnaires de planifier les services, d'affecter les véhicules et conducteurs aux missions, et d'optimiser l'utilisation de la flotte. C'est le module le plus demandé par les entreprises de transport.

2.1 Exigences Fonctionnelles

Exigence

EF-PL01 — Création de planning : Le gestionnaire doit pouvoir créer des plannings hebdomadaires ou mensuels définissant les plages de service de chaque conducteur et véhicule.

Exigence

EF-PL02 — Affectation intelligente : Le système doit proposer automatiquement les affectations optimales en tenant compte de la disponibilité des conducteurs, de l'état des véhicules et des contraintes légales (temps de conduite maximum).

Exigence

EF-PL03 — Détection de conflits : Le système doit détecter et signaler les conflits d'affectation (même conducteur ou véhicule affecté à deux missions simultanées).

Exigence

EF-PL04 — Suivi en temps réel : Le gestionnaire doit visualiser en temps réel l'état d'avancement de chaque affectation (En attente, En cours, Terminée, Annulée).

Exigence

EF-PL05 — Historique des affectations : L'historique complet des affectations doit être consultable avec filtres par période, conducteur, véhicule ou statut.

2.2 Modèle de Données

Listing 2.1 : Tables du module Planification

```

1  -- Planning de service (semaine ou mois)
2  CREATE TABLE IF NOT EXISTS fleet.schedules (
3      id                UUID PRIMARY KEY,
4      fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
5      manager_id        UUID NOT NULL REFERENCES fleet.fleet_managers(
6          user_id),
7      title             VARCHAR(255) NOT NULL,
8      period_type       VARCHAR(20) NOT NULL
9                      CHECK (period_type IN ('DAILY', 'WEEKLY', 'MONTHLY')),
10     start_date         DATE NOT NULL,
11     end_date           DATE NOT NULL,
12     status             VARCHAR(20) DEFAULT 'DRAFT'
13                     CHECK (status IN ('DRAFT', 'PUBLISHED', 'ARCHIVED')),
14     notes              TEXT,
15     created_at         TIMESTAMP DEFAULT now(),
16     created_by         UUID REFERENCES fleet.fleet_managers(user_id)
17 );
18
19 -- Affectation : un conducteur + un vehicule pour une plage horaire
20 CREATE TABLE IF NOT EXISTS fleet.assignments (
21     id                UUID PRIMARY KEY,
22     schedule_id       UUID REFERENCES fleet.schedules(id) ON DELETE
23                     CASCADE,
24     fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
25     vehicle_id        UUID NOT NULL REFERENCES fleet.vehicles(id),
26     driver_id         UUID NOT NULL REFERENCES fleet.drivers(user_id),
27     mission_id        UUID, -- FK vers fleet.missions (Module 7)
28     start_datetime    TIMESTAMP NOT NULL,
29     end_datetime      TIMESTAMP NOT NULL,

```

```
28     status          VARCHAR(30) DEFAULT 'PENDING'
29                     CHECK (status IN ('PENDING', 'IN_PROGRESS',
30                                     'COMPLETED', 'CANCELLED', 'NO_SHOW'
31                                     )),
32     start_location   VARCHAR(255),
33     end_location     VARCHAR(255),
34     estimated_km     NUMERIC(10,2),
35     actual_km        NUMERIC(10,2),
36     notes            TEXT,
37     created_at       TIMESTAMP DEFAULT now(),
38     CONSTRAINT chk_dates CHECK (end_datetime > start_datetime)
39 );
40 CREATE INDEX IF NOT EXISTS idx_assignments_vehicle
41     ON fleet.assignments(vehicle_id, start_datetime);
42 CREATE INDEX IF NOT EXISTS idx_assignments_driver
43     ON fleet.assignments(driver_id, start_datetime);
44 CREATE INDEX IF NOT EXISTS idx_assignments_schedule
45     ON fleet.assignments(schedule_id);
```

2.3 Endpoints REST Proposés

Méthode URL		Rôle	HTTP
POST	/api/v1/schedules	MANAGER	201
GET	/api/v1/schedules	MANAGER	200
GET	/api/v1/schedules/{id}	MANAGER	200
PUT	/api/v1/schedules/{id}/publish	MANAGER	200
DELETE	/api/v1/schedules/{id}	MANAGER	204
POST	/api/v1/assignments	MANAGER	201
GET	/api/v1/assignments	MANAGER	200
GET	/api/v1/assignments/conflicts	MANAGER	200
PATCH	/api/v1/assignments/{id}/status	MANAGER/DRIVER	200
GET	/api/v1/assignments/driver/{id}/driver	DRIVER	200
GET	/api/v1/assignments/vehicle/{id}/availability	MANAGER	200

TAB. 2.1 : Endpoints du module Planification

Chapitre 3

Module 2 — Gestion des Contrats et Documents Légaux

Résumé du Module

Priorité : P1 — Critique **Effort : Medium (2 semaines)** **Impact : Critique**

La conformité légale est non-négociable dans le transport. Ce module gère les documents des véhicules (assurance, visite technique, carte grise) et des conducteurs (permis, visite médicale), avec alertes automatiques avant expiration.

3.1 Exigences Fonctionnelles

Exigence

EF-DOC01 — Gestion documentaire véhicule : Le système doit gérer les documents légaux obligatoires : assurance, carte grise, visite technique, vignette, autorisation de transport.

Exigence

EF-DOC02 — Gestion documentaire conducteur : Le système doit gérer : permis de conduire (catégories), visite médicale, carte professionnelle, contrat de travail.

Exigence

EF-DOC03 — Alertes d'expiration : Le système doit envoyer des notifications automatiques à J-30, J-15 et J-7 avant l'expiration de tout document. Un document expiré doit bloquer l'affectation du véhicule ou conducteur concerné.

Exigence

EF-DOC04 — Stockage des fichiers : Les documents numérisés (PDF, JPEG) doivent pouvoir être attachés à chaque entrée documentaire.

Exigence

EF-DOC05 — Tableau de bord conformité : Une vue synthétique doit afficher le taux de conformité documentaire de la flotte (nombre de documents valides / total).

3.2 Modèle de Données

Listing 3.1 : Tables du module Documents

```

1  -- Documents des vehicules
2  CREATE TABLE IF NOT EXISTS fleet.vehicle_documents (
3      id                UUID PRIMARY KEY,
4      vehicle_id        UUID NOT NULL REFERENCES fleet.vehicles(id)
5                          ON DELETE CASCADE,
6      doc_type           VARCHAR(50) NOT NULL
7                          CHECK (doc_type IN (
8                              'INSURANCE',          -- Assurance
9                              'REGISTRATION',       -- Carte grise
10                             'TECHNICAL_CONTROL',  -- Visite technique
11                             'TAX_STICKER',        -- Vignette
12                             'TRANSPORT_PERMIT',   -- Autorisation transport
13                             'OTHER'
14                         )),
15      doc_number         VARCHAR(100),
16      issuer             VARCHAR(255),
17      issue_date         DATE,
18      expiry_date        DATE NOT NULL,
19      file_url           VARCHAR(500),
20      status             VARCHAR(20) DEFAULT 'VALID'
21                          CHECK (status IN ('VALID', 'EXPIRING_SOON',
22                                              'EXPIRED', 'PENDING_RENEWAL')),
23      notes              TEXT,
24      created_at         TIMESTAMP DEFAULT now(),
25      updated_at         TIMESTAMP DEFAULT now()
26 );
27
28 -- Documents des conducteurs
29 CREATE TABLE IF NOT EXISTS fleet.driver_documents (
30     id                UUID PRIMARY KEY,
31     driver_id         UUID NOT NULL REFERENCES fleet.drivers(user_id)

```

```

32         ON DELETE CASCADE,
33     doc_type          VARCHAR(50) NOT NULL
34     CHECK (doc_type IN (
35         'DRIVING_LICENSE', -- Permis de conduire
36         'MEDICAL_CERT',    -- Visite medicale
37         'PROFESSIONAL_CARD', -- Carte professionnelle
38         'WORK_CONTRACT',   -- Contrat de travail
39         'ID_CARD',         -- Carte nationale
40         'OTHER'
41     )),
42     doc_number        VARCHAR(100),
43     license_categories VARCHAR(50), -- Ex: B,C,D pour le permis
44     issuer            VARCHAR(255),
45     issue_date        DATE,
46     expiry_date       DATE,
47     file_url          VARCHAR(500),
48     status            VARCHAR(20) DEFAULT 'VALID'
49     CHECK (status IN ('VALID', 'EXPIRING_SOON',
50         'EXPIRED', 'PENDING_RENEWAL')),
51     notes             TEXT,
52     created_at        TIMESTAMP DEFAULT now()
53 );
54
55 -- Historique des alertes d'expiration envoyees
56 CREATE TABLE IF NOT EXISTS fleet.document_alerts (
57     id                UUID PRIMARY KEY,
58     document_id       UUID NOT NULL,
59     document_type     VARCHAR(20) NOT NULL CHECK (document_type IN
60         ('VEHICLE', 'DRIVER')),
61     alert_type        VARCHAR(20) NOT NULL CHECK (alert_type IN
62         ('J30', 'J15', 'J7', 'EXPIRED')),
63     sent_at          TIMESTAMP DEFAULT now(),
64     recipient_id      UUID NOT NULL
65 );
66
67 CREATE INDEX IF NOT EXISTS idx_vdoc_expiry
68     ON fleet.vehicle_documents(expiry_date, status);
69 CREATE INDEX IF NOT EXISTS idx_ddoc_expiry
70     ON fleet.driver_documents(expiry_date, status);

```

3.3 Endpoints REST Proposés

Méthode	URL	Rôle	HTTP
POST	/api/v1/vehicles/{id}/documents	MANAGER	201
GET	/api/v1/vehicles/{id}/documents	MANAGER	200
PUT	/api/v1/vehicles/{id}/documents/{docId}	MANAGER	200
DELETE	/api/v1/vehicles/{id}/documents/{docId}	MANAGER	204
POST	/api/v1/drivers/{id}/documents	MANAGER	201
GET	/api/v1/drivers/{id}/documents	MANAGER	200
GET	/api/v1/documents/expiring	MANAGER	200
GET	/api/v1/documents/compliance-report	MANAGER	200
GET	/api/v1/documents/expired	MANAGER	200

TAB. 3.1 : Endpoints du module Documents

Note Experte

Un job planifié (Spring `@Scheduled` ou Quartz) doit s'exécuter chaque nuit pour scanner les documents dont l'expiration approche et déclencher les notifications via le port `SendNotificationPort` existant. Ce job met également à jour le champ `status` des documents.

Chapitre 4

Module 3 — Tableau de Bord KPI et Rapports Avancés

Résumé du Module

Priorité : **P1 — Critique**

Effort : **Medium (2 semaines)**

Impact : **Élevé**

Les KPIs sont le cœur de la valeur perçue par le gestionnaire. Ce module calcule et stocke les indicateurs de performance clés de la flotte, permettant le pilotage opérationnel et la prise de décision.

4.1 KPIs Métier Identifiés

Un expert en gestion de flotte suit en permanence les indicateurs suivants :

KPI	Formule	Interprétation
Taux de disponibilité	$\frac{\text{Véhicules disponibles}}{\text{Total}}$	$\geq 85\%$ = bonne flotte
Taux d'utilisation	$\frac{\text{Km parcourus}}{\text{Km potentiels}}$	Optimisation des ressources
Coût par km	$\frac{\text{Dépenses totales}}{\text{Km totaux}}$	Rentabilité opérationnelle
Consommation L/100km	$\frac{\text{Litres}}{\text{Km} / 100}$	Efficacité énergétique
MTBF	$\frac{\text{Temps entre pannes}}{\text{Nb véhicules}}$	Fiabilité de la flotte
Coût maintenance/véhicule	$\frac{\text{Coût maintenances}}{\text{Nb véhicules}}$	Vieillessement du parc
Taux d'incidents	$\frac{\text{Incidents}}{1000 \text{ km}}$	Sécurité routière
Score moyen conducteurs	Moyenne des scores	Qualité de conduite

KPI	Formule	Interprétation
Taux de conformité docs	Docs valides / Total docs	Conformité légale
Revenus par véhicule	CA total / Nb véhicules	Rentabilité du parc

TAB. 4.1 : KPIs métier de la gestion de flotte

4.2 Exigences Fonctionnelles

Exigence

EF-KPI01 — Calcul périodique : Les KPIs doivent être calculés et stockés quotidiennement, hebdomadairement et mensuellement via un job planifié.

Exigence

EF-KPI02 — Comparaison temporelle : Le gestionnaire doit pouvoir comparer les KPIs sur deux périodes (ex : ce mois vs mois précédent, cette année vs année précédente).

Exigence

EF-KPI03 — Granularité : Les KPIs doivent être disponibles au niveau flotte, véhicule individuel et conducteur individuel.

Exigence

EF-KPI04 — Export : Les rapports doivent être exportables en CSV et PDF avec graphiques.

4.3 Modèle de Données

Listing 4.1 : Table des instantanés KPI

```
1  -- Stockage des KPIs calculés périodiquement
2  CREATE TABLE IF NOT EXISTS fleet.kpi_snapshots (
3      id                UUID PRIMARY KEY,
4      fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
5      entity_type       VARCHAR(20) NOT NULL
6      CHECK (entity_type IN ('FLEET', 'VEHICLE', 'DRIVER'))
7  );
```

```

7      entity_id      UUID NOT NULL,  -- ID de la flotte, vehicule ou
      conducteur
8      period_type    VARCHAR(10) NOT NULL
9                      CHECK (period_type IN ('DAILY', 'WEEKLY', 'MONTHLY'))
10
11      period_start    DATE NOT NULL,
12      period_end      DATE NOT NULL,
13
14      -- KPIs operationnels
15      total_km         NUMERIC(12,2) DEFAULT 0,
16      total_trips       INT DEFAULT 0,
17      total_driving_hours NUMERIC(8,2) DEFAULT 0,
18      availability_rate NUMERIC(5,2),  -- Pourcentage
19
20      -- KPIs financiers
21      total_fuel_cost   NUMERIC(12,2) DEFAULT 0,
22      total_fuel_liters  NUMERIC(10,2) DEFAULT 0,
23      total_maintenance_cost NUMERIC(12,2) DEFAULT 0,
24      total_incident_cost NUMERIC(12,2) DEFAULT 0,
25      total_expenses     NUMERIC(12,2) DEFAULT 0,
26      cost_per_km       NUMERIC(8,4),
27      fuel_per_100km     NUMERIC(6,2),
28
29      -- KPIs securite
30      total_incidents   INT DEFAULT 0,
31      incident_rate      NUMERIC(6,4),  -- Incidents / 1000 km
32      avg_driver_score   NUMERIC(5,2),
33
34      -- KPIs conformite
35      doc_compliance_rate NUMERIC(5,2),
36
37      calculated_at      TIMESTAMP DEFAULT now(),
38
39      UNIQUE (entity_type, entity_id, period_type, period_start)
40 );
41
42 CREATE INDEX IF NOT EXISTS idx_kpi_entity
43   ON fleet.kpi_snapshots(entity_type, entity_id, period_start DESC);
44
45 CREATE INDEX IF NOT EXISTS idx_kpi_fleet
46   ON fleet.kpi_snapshots(fleet_id, period_type, period_start DESC);

```

4.4 Endpoints REST Proposés

Méthode	URL	Rôle	HTTP
GET	/api/v1/kpis/fleet/{id}	MANAGER	200
GET	/api/v1/kpis/vehicle/{id}	MANAGER	200
GET	/api/v1/kpis/driver/{id}	MANAGER	200
GET	/api/v1/kpis/fleet/{id}/compare	MANAGER	200
GET	/api/v1/kpis/fleet/{id}/top-vehicles	MANAGER	200
GET	/api/v1/kpis/fleet/{id}/top-drivers	MANAGER	200
GET	/api/v1/kpis/fleet/{id}/export	MANAGER	200
POST	/api/v1/kpis/recalculate	ADMIN	202

TAB. 4.2 : Endpoints du module KPI

Chapitre 5

Module 4 — Gestion des Dépenses et Budget

Résumé du Module

Priorité : P2 — Haute **Effort :** Medium (2 semaines) **Impact :** Élevé

La maîtrise des coûts est l'enjeu numéro un des gestionnaires de flotte. Ce module centralise toutes les dépenses opérationnelles et permet de les comparer à des budgets prévisionnels.

5.1 Exigences Fonctionnelles

Exigence

EF-DEP01 — Catégorisation des dépenses : Toute dépense doit être catégorisée (carburant, maintenance, assurance, péages, amendes, lavage, pneus, etc.) pour permettre une analyse par poste de coût.

Exigence

EF-DEP02 — Saisie multi-acteurs : Les conducteurs peuvent saisir leurs dépenses terrain (péages, lavage) depuis l'application mobile. Le gestionnaire valide ou rejette ces dépenses.

Exigence

EF-DEP03 — Gestion budgétaire : Le gestionnaire doit pouvoir définir des budgets mensuels ou annuels par flotte et par catégorie. Le système doit alerter lorsque 80% du budget est consommé.

Exigence

EF-DEP04 — Justificatifs : Chaque dépense doit pouvoir être accompagnée d'un justificatif (photo de reçu, facture PDF).

5.2 Modèle de Données

Listing 5.1 : Tables du module Dépenses et Budget

```

1  -- Categories de dépenses (configurables par le gestionnaire)
2  CREATE TABLE IF NOT EXISTS fleet.expense_categories (
3      id          UUID PRIMARY KEY,
4      fleet_id    UUID REFERENCES fleet.fleets(id) ON DELETE CASCADE,
5      name        VARCHAR(100) NOT NULL,
6      code        VARCHAR(30) NOT NULL,
7      color       VARCHAR(7), -- Code couleur hex pour l'UI
8      is_system   BOOLEAN DEFAULT false, -- Categories systeme non
          supprimables
9      UNIQUE (fleet_id, code)
10 );
11
12 -- Dépenses opérationnelles
13 CREATE TABLE IF NOT EXISTS fleet.expenses (
14     id          UUID PRIMARY KEY,
15     fleet_id    UUID NOT NULL REFERENCES fleet.fleets(id),
16     vehicle_id  UUID REFERENCES fleet.vehicles(id) ON DELETE SET
          NULL,
17     driver_id   UUID REFERENCES fleet.drivers(user_id) ON DELETE
          SET NULL,
18     category_id UUID NOT NULL REFERENCES fleet.expense_categories(
          id),
19     amount      NUMERIC(12,2) NOT NULL CHECK (amount > 0),
20     currency    VARCHAR(3) DEFAULT 'XAF',
21     expense_date DATE NOT NULL,
22     description VARCHAR(500),
23     receipt_url  VARCHAR(500),
24     status      VARCHAR(20) DEFAULT 'PENDING'
          CHECK (status IN ('PENDING', 'APPROVED',
25                          'REJECTED', 'REIMBURSED')),
26
27     submitted_by UUID NOT NULL,
28     approved_by  UUID,
29     approved_at  TIMESTAMP,
30     rejection_reason TEXT,
31     created_at   TIMESTAMP DEFAULT now()
32 );
33
34 -- Budgets previsionnels

```

```
35 CREATE TABLE IF NOT EXISTS fleet.budgets (  
36     id                UUID PRIMARY KEY,  
37     fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),  
38     category_id       UUID REFERENCES fleet.expense_categories(id),  
39     period_type       VARCHAR(10) NOT NULL  
40     CHECK (period_type IN ('MONTHLY', 'QUARTERLY', '  
                        ANNUAL')),  
41     period_start      DATE NOT NULL,  
42     period_end        DATE NOT NULL,  
43     amount            NUMERIC(12,2) NOT NULL CHECK (amount > 0),  
44     currency          VARCHAR(3) DEFAULT 'XAF',  
45     alert_threshold   NUMERIC(5,2) DEFAULT 80.0,  -- % de consommation  
46     created_at        TIMESTAMP DEFAULT now(),  
47     UNIQUE (fleet_id, category_id, period_type, period_start)  
48 );  
49  
50 CREATE INDEX IF NOT EXISTS idx_expenses_fleet_date  
51     ON fleet.expenses(fleet_id, expense_date DESC);  
52 CREATE INDEX IF NOT EXISTS idx_expenses_vehicle  
53     ON fleet.expenses(vehicle_id, expense_date DESC);
```

5.3 Endpoints REST Proposés

Méthode URL		Rôle	HTTP
POST	/api/v1/expenses	MANAGER/DRIVER	201
GET	/api/v1/expenses	MANAGER	200
PATCH	/api/v1/expenses/{id}/approve	MANAGER	200
PATCH	/api/v1/expenses/{id}/reject	MANAGER	200
GET	/api/v1/expenses/summary	MANAGER	200
GET	/api/v1/expenses/by-category	MANAGER	200
POST	/api/v1/budgets	MANAGER	201
GET	/api/v1/budgets	MANAGER	200
GET	/api/v1/budgets/consumption	MANAGER	200
GET	/api/v1/expense-categories	MANAGER	200
POST	/api/v1/expense-categories	MANAGER	201

TAB. 5.1 : Endpoints du module Dépenses

Chapitre 6

Module 5 — Scoring Conducteur et Éco-conduite

Résumé du Module

Priorité : P2 — Haute **Effort :** Large (3 semaines) **Impact :** Élevé

Le scoring conducteur est un différenciateur majeur. Il permet de réduire les accidents, la consommation de carburant et l'usure des véhicules en analysant le comportement de conduite en temps réel.

6.1 Exigences Fonctionnelles

Exigence

EF-SCO01 — Détection d'événements : Le système doit détecter et enregistrer les événements de conduite dangereux : excès de vitesse, freinage brusque, accélération brutale, virage serré, conduite de nuit prolongée.

Exigence

EF-SCO02 — Calcul du score : Un score de conduite sur 100 doit être calculé pour chaque conducteur à la fin de chaque trajet et agrégé mensuellement. La formule doit être configurable par le gestionnaire.

Exigence

EF-SCO03 — Classement : Un classement des conducteurs par score doit être disponible pour encourager la compétition positive et identifier les conducteurs nécessitant une formation.

Exigence

EF-SCO04 — Feedback conducteur : Le conducteur doit recevoir son score après chaque trajet avec le détail des événements détectés, pour s'améliorer.

Exigence

EF-SCO05 — Corrélation coûts : Le système doit corréler le score de conduite avec les coûts de maintenance et de carburant pour démontrer le ROI de l'éco-conduite.

6.2 Algorithme de Scoring

Le score est calculé sur 100 points, avec des pénalités par événement :

Événement	Pénalité	Seuil de détection
Excès de vitesse léger	-2 pts	> limite + 10 km/h
Excès de vitesse grave	-5 pts	> limite + 30 km/h
Freinage brusque	-3 pts	Décélération > 0.4g
Accélération brutale	-2 pts	Accélération > 0.3g
Virage serré	-2 pts	Accélération latérale > 0.3g
Conduite nocturne (> 4h)	-3 pts	Entre 22h et 5h
Téléphone au volant	-10 pts	Détection via accéléromètre

TAB. 6.1 : Barème de pénalités du scoring conducteur

6.3 Modèle de Données

Listing 6.1 : Tables du module Scoring Conducteur

```

1  -- Evenements de conduite detectes pendant un trajet
2  CREATE TABLE IF NOT EXISTS fleet.driving_events (
3      id                UUID PRIMARY KEY,
4      trip_id           UUID NOT NULL REFERENCES fleet.trips(id)
5                          ON DELETE CASCADE,
6      driver_id         UUID NOT NULL REFERENCES fleet.drivers(user_id),
7      vehicle_id        UUID NOT NULL REFERENCES fleet.vehicles(id),
8      event_type        VARCHAR(50) NOT NULL CHECK (event_type IN (
9                          'SPEEDING_MINOR',
10                         'SPEEDING_MAJOR',
11                         'HARSH_BRAKING',
12                         'HARSH_ACCELERATION',
13                         'SHARP_CORNERING',

```

```

14         'NIGHT_DRIVING',
15         'PHONE_USAGE',
16         'IDLE_ENGINE'
17     )),
18     severity      VARCHAR(20) DEFAULT 'MEDIUM'
19                 CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH')),
20     detected_at   TIMESTAMP NOT NULL,
21     longitude     NUMERIC(10,7),
22     latitude      NUMERIC(10,7),
23     speed_kmh     NUMERIC(6,2),
24     speed_limit_kmh NUMERIC(6,2),
25     penalty_points NUMERIC(4,2) NOT NULL DEFAULT 0,
26     details       JSONB -- Donnees brutes de l'accelerometre
27 );
28
29 -- Scores de conduite calcules par trajet et par periode
30 CREATE TABLE IF NOT EXISTS fleet.driver_scores (
31     id            UUID PRIMARY KEY,
32     driver_id     UUID NOT NULL REFERENCES fleet.drivers(user_id),
33     fleet_id      UUID NOT NULL REFERENCES fleet.fleets(id),
34     score_type    VARCHAR(20) NOT NULL
35                 CHECK (score_type IN ('TRIP', 'DAILY',
36                                     'WEEKLY', 'MONTHLY')),
37     reference_id  UUID, -- trip_id si TRIP, null sinon
38     period_start  DATE NOT NULL,
39     period_end    DATE NOT NULL,
40     base_score    NUMERIC(5,2) DEFAULT 100,
41     total_penalty NUMERIC(5,2) DEFAULT 0,
42     final_score   NUMERIC(5,2) NOT NULL, -- base - penalites
43     total_km      NUMERIC(10,2),
44     event_count   INT DEFAULT 0,
45     rank_in_fleet INT, -- Classement dans la flotte
46     calculated_at TIMESTAMP DEFAULT now(),
47     UNIQUE (driver_id, score_type, period_start)
48 );
49
50 CREATE INDEX IF NOT EXISTS idx_driving_events_trip
51     ON fleet.driving_events(trip_id);
52 CREATE INDEX IF NOT EXISTS idx_driving_events_driver
53     ON fleet.driving_events(driver_id, detected_at DESC);
54 CREATE INDEX IF NOT EXISTS idx_driver_scores_fleet
55     ON fleet.driver_scores(fleet_id, score_type, period_start DESC);

```

6.4 Endpoints REST Proposés

Méthode	URL	Rôle	HTTP
POST	/api/v1/trips/{id}/driving-events	DRIVER	201
GET	/api/v1/drivers/{id}/score	MANAGER/DRIVER	200
GET	/api/v1/drivers/{id}/score/history	MANAGER	200
GET	/api/v1/drivers/{id}/driving-events	MANAGER	200
GET	/api/v1/fleets/{id}/driver-ranking	MANAGER	200
GET	/api/v1/fleets/{id}/score-summary	MANAGER	200

TAB. 6.2 : Endpoints du module Scoring

Chapitre 7

Module 6 — Maintenance Préventive Intelligente

Résumé du Module

Priorité : P2 — Haute **Effort :** Medium (2 semaines) **Impact :** Élevé

La maintenance préventive réduit les pannes de 40% et les coûts de maintenance de 25% selon les études sectorielles. Ce module planifie automatiquement les maintenances selon le kilométrage, le temps ou les données de télémétrie.

7.1 Exigences Fonctionnelles

Exigence

EF-MP01 — Plans de maintenance : Le gestionnaire doit pouvoir définir des plans de maintenance pour chaque type de véhicule, spécifiant les tâches à effectuer selon des déclencheurs (kilométrage, durée, date calendaire).

Exigence

EF-MP02 — Déclenchement automatique : Le système doit créer automatiquement des tâches de maintenance lorsque les seuils sont atteints (ex : vidange tous les 10 000 km ou 6 mois).

Exigence

EF-MP03 — Alertes préventives : Des notifications doivent être envoyées au gestionnaire et au conducteur lorsqu'une maintenance approche (à 500 km ou 15 jours du seuil).

Exigence

EF-MP04 — Suivi des tâches : Chaque tâche de maintenance doit suivre un cycle de vie : Planifiée → En cours → Terminée → Annulée.

7.2 Modèle de Données

Listing 7.1 : Tables du module Maintenance Préventive

```

1  -- Plans de maintenance (modeles reutilisables)
2  CREATE TABLE IF NOT EXISTS fleet.maintenance_plans (
3      id                UUID PRIMARY KEY,
4      fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
5      vehicle_type_id   UUID REFERENCES fleet.vehicle_types(id),
6      name              VARCHAR(255) NOT NULL,
7      description       TEXT,
8      is_active         BOOLEAN DEFAULT true,
9      created_at        TIMESTAMP DEFAULT now()
10 );
11
12 -- Taches dans un plan de maintenance
13 CREATE TABLE IF NOT EXISTS fleet.maintenance_tasks (
14     id                UUID PRIMARY KEY,
15     plan_id           UUID NOT NULL REFERENCES fleet.maintenance_plans(id
16 )
17     ON DELETE CASCADE,
18     vehicle_id        UUID REFERENCES fleet.vehicles(id) ON DELETE
19     CASCADE,
20     task_name         VARCHAR(255) NOT NULL,
21     task_type         VARCHAR(50) NOT NULL CHECK (task_type IN (
22         'OIL_CHANGE',      -- Vidange
23         'TIRE_ROTATION',   -- Rotation pneus
24         'BRAKE_CHECK',     -- Controle freins
25         'FILTER_CHANGE',  -- Changement filtres
26         'BELT_CHECK',      -- Controle courroies
27         'BATTERY_CHECK',  -- Controle batterie
28         'FULL_SERVICE',   -- Revision complete
29         'CUSTOM'          -- Personnalise
30     )),
31     trigger_type      VARCHAR(20) NOT NULL
32     CHECK (trigger_type IN ('KM', 'DAYS', 'DATE')),
33     trigger_value      NUMERIC(10,2) NOT NULL, -- Km ou jours
34     last_done_km       NUMERIC(12,2),
35     last_done_date     DATE,
36     next_due_km        NUMERIC(12,2),
37     next_due_date      DATE,
38     status             VARCHAR(20) DEFAULT 'SCHEDULED '

```

```
37         CHECK (status IN ('SCHEDULED','OVERDUE',
38                           'IN_PROGRESS','DONE','CANCELLED'))
39     ),
40     estimated_cost NUMERIC(10,2),
41     actual_cost    NUMERIC(10,2),
42     assigned_to    VARCHAR(255), -- Garage ou technicien
43     completed_at   TIMESTAMP,
44     notes          TEXT,
45     created_at     TIMESTAMP DEFAULT now()
46 );
47
48 CREATE INDEX IF NOT EXISTS idx_maint_tasks_vehicle
49     ON fleet.maintenance_tasks(vehicle_id, status);
50
51 CREATE INDEX IF NOT EXISTS idx_maint_tasks_due
52     ON fleet.maintenance_tasks(next_due_date, status);
```

7.3 Endpoints REST Proposés

Méthode URL		Rôle	HTTP
POST	/api/v1/maintenance-plans	MANAGER	201
GET	/api/v1/maintenance-plans	MANAGER	200
POST	/api/v1/maintenance-tasks	MANAGER	201
GET	/api/v1/maintenance-tasks	MANAGER	200
GET	/api/v1/maintenance-tasks/overdue	MANAGER	200
GET	/api/v1/maintenance-tasks/upcoming	MANAGER	200
PATCH	/api/v1/maintenance-tasks/{id}/complete	MANAGER	200
GET	/api/v1/vehicles/{id}/maintenance-schedule	MANAGER	200

TAB. 7.1 : Endpoints du module Maintenance Préventive

Chapitre 8

Module 7 — Gestion des Clients et Missions

Résumé du Module

Priorité : P3 — Normale **Effort :** Large (3-4 semaines) **Impact :** Très élevé (monétisation)

Ce module transforme FleetMan d'un outil de supervision interne en une plateforme de gestion commerciale complète. Il permet de gérer les clients, les missions de transport et la facturation, ouvrant la voie à la monétisation directe de la plateforme.

8.1 Exigences Fonctionnelles

Exigence

EF-CLI01 — Gestion du portefeuille clients : Le gestionnaire doit pouvoir créer et gérer ses clients (entreprises ou particuliers) avec leurs coordonnées, contrats-cadres et historique de prestations.

Exigence

EF-CLI02 — Création de missions : Une mission représente une prestation de transport commandée par un client. Elle doit inclure : client, type de transport, point de départ, point d'arrivée, date/heure, véhicule requis, tarif.

Exigence

EF-CLI03 — Affectation automatique : Lors de la création d'une mission, le système doit proposer les véhicules et conducteurs disponibles correspondant aux critères (type, capacité, zone géographique).

Exigence

EF-CLI04 — Facturation : À la clôture d’une mission, le système doit générer automatiquement une facture basée sur le tarif convenu, la distance réelle parcourue et les éventuels suppléments.

Exigence

EF-CLI05 — Suivi client : Le client doit pouvoir suivre sa mission en temps réel via un lien de suivi public (sans authentification), affichant la position du véhicule et l’heure d’arrivée estimée.

Exigence

EF-CLI06 — Historique et statistiques : L’historique complet des missions par client doit être disponible avec les KPIs associés (CA généré, nombre de missions, distance totale).

8.2 Modèle de Données

Listing 8.1 : Tables du module Clients et Missions

```

1  -- Clients de l'entreprise de transport
2  CREATE TABLE IF NOT EXISTS fleet.clients (
3      id                UUID PRIMARY KEY,
4      fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
5      client_type       VARCHAR(20) NOT NULL
6                          CHECK (client_type IN ('INDIVIDUAL', 'COMPANY')),
7      name              VARCHAR(255) NOT NULL,
8      email             VARCHAR(255),
9      phone             VARCHAR(50),
10     address           TEXT,
11     city              VARCHAR(100),
12     tax_number        VARCHAR(100),  -- Numero contribuable
13     contact_person    VARCHAR(255),  -- Pour les entreprises
14     notes             TEXT,
15     is_active         BOOLEAN DEFAULT true,
16     created_at        TIMESTAMP DEFAULT now()
17 );
18
19 -- Missions de transport
20 CREATE TABLE IF NOT EXISTS fleet.missions (
21     id                UUID PRIMARY KEY,
22     fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
23     client_id         UUID NOT NULL REFERENCES fleet.clients(id),
24     assignment_id     UUID REFERENCES fleet.assignments(id),

```



```

25 mission_ref          VARCHAR(50) UNIQUE NOT NULL, -- Reference unique
26 mission_type         VARCHAR(50) NOT NULL CHECK (mission_type IN (
27     'FREIGHT',        -- Transport de marchandises
28     'PASSENGER',      -- Transport de personnes
29     'SCHOOL_RUN',     -- Transport scolaire
30     'MEDICAL',        -- Transport medical
31     'VIP',            -- Transport VIP
32     'DELIVERY',       -- Livraison
33     'OTHER'
34     )),
35 status               VARCHAR(30) DEFAULT 'PENDING' CHECK (status IN (
36     'PENDING',        -- En attente de
37                        confirmation
38     'CONFIRMED',      -- Confirmée
39     'IN_PROGRESS',    -- En cours
40     'COMPLETED',     -- Terminée
41     'CANCELLED',      -- Annulée
42     'DISPUTED'        -- En litige
43     )),
44 -- Localisation
45 origin_address        VARCHAR(500) NOT NULL,
46 origin_lat            NUMERIC(10,7),
47 origin_lng            NUMERIC(10,7),
48 destination_address   VARCHAR(500) NOT NULL,
49 destination_lat       NUMERIC(10,7),
50 destination_lng       NUMERIC(10,7),
51 -- Planification
52 scheduled_start       TIMESTAMP NOT NULL,
53 scheduled_end         TIMESTAMP,
54 actual_start          TIMESTAMP,
55 actual_end            TIMESTAMP,
56 -- Tarification
57 base_rate             NUMERIC(12,2),
58 rate_type             VARCHAR(20) CHECK (rate_type IN
59     ('FIXED', 'PER_KM', 'HOURLY')),
60 estimated_km          NUMERIC(10,2),
61 actual_km            NUMERIC(10,2),
62 estimated_amount      NUMERIC(12,2),
63 actual_amount        NUMERIC(12,2),
64 currency             VARCHAR(3) DEFAULT 'XAF',
65 -- Suivi public
66 tracking_token        VARCHAR(100) UNIQUE, -- Token pour lien public
67 tracking_expires_at   TIMESTAMP,
68 -- Metadata
69 cargo_description     TEXT,
70 cargo_weight_kg       NUMERIC(8,2),
71 passenger_count       INT,

```

```

71     special_instructions TEXT,
72     cancellation_reason TEXT,
73     created_at          TIMESTAMP DEFAULT now()
74 );
75
76 -- Factures
77 CREATE TABLE IF NOT EXISTS fleet.invoices (
78     id                UUID PRIMARY KEY,
79     fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
80     client_id         UUID NOT NULL REFERENCES fleet.clients(id),
81     mission_id        UUID REFERENCES fleet.missions(id),
82     invoice_number    VARCHAR(50) UNIQUE NOT NULL,
83     invoice_date      DATE NOT NULL DEFAULT CURRENT_DATE,
84     due_date          DATE NOT NULL,
85     status            VARCHAR(20) DEFAULT 'DRAFT' CHECK (status IN (
86         'DRAFT', 'SENT', 'PAID', 'OVERDUE', 'CANCELLED'
87     )),
88     subtotal          NUMERIC(12,2) NOT NULL,
89     tax_rate          NUMERIC(5,2) DEFAULT 19.25, -- TVA Cameroun
90     tax_amount        NUMERIC(12,2),
91     total_amount      NUMERIC(12,2) NOT NULL,
92     currency          VARCHAR(3) DEFAULT 'XAF',
93     payment_method    VARCHAR(50),
94     paid_at           TIMESTAMP,
95     notes             TEXT,
96     pdf_url           VARCHAR(500),
97     created_at        TIMESTAMP DEFAULT now()
98 );
99
100 CREATE INDEX IF NOT EXISTS idx_missions_client
101     ON fleet.missions(client_id, scheduled_start DESC);
102 CREATE INDEX IF NOT EXISTS idx_missions_fleet_status
103     ON fleet.missions(fleet_id, status, scheduled_start DESC);
104 CREATE INDEX IF NOT EXISTS idx_missions_tracking
105     ON fleet.missions(tracking_token) WHERE tracking_token IS NOT NULL;
106 CREATE INDEX IF NOT EXISTS idx_invoices_client
107     ON fleet.invoices(client_id, invoice_date DESC);

```

8.3 Endpoints REST Proposés

Méthode URL		Rôle	HTTP
POST	/api/v1/clients	MANAGER	201
GET	/api/v1/clients	MANAGER	200

Méthode	URL	Rôle	HTTP
GET	/api/v1/clients/{id}	MANAGER	200
PUT	/api/v1/clients/{id}	MANAGER	200
GET	/api/v1/clients/{id}/missions	MANAGER	200
GET	/api/v1/clients/{id}/stats	MANAGER	200
POST	/api/v1/missions	MANAGER	201
GET	/api/v1/missions	MANAGER	200
GET	/api/v1/missions/{id}	MANAGER	200
PATCH	/api/v1/missions/{id}/status	MANAGER	200
GET	/api/v1/missions/{id}/available-vehicles	MANAGER	200
GET	/api/v1/track/{token}	PUBLIC	200
POST	/api/v1/invoices/generate/{missionId}	MANAGER	201
GET	/api/v1/invoices	MANAGER	200
PATCH	/api/v1/invoices/{id}/mark-paid	MANAGER	200
GET	/api/v1/invoices/{id}/pdf	MANAGER	200

TAB. 8.1 : Endpoints du module Clients et Missions

Note Experte

L'endpoint `GET /api/v1/track/{token}` est public (sans authentification). Il retourne uniquement la position en temps réel du véhicule et l'heure d'arrivée estimée, sans exposer de données sensibles. Le token expire automatiquement à la fin de la mission.

Chapitre 9

Module 8 — Alertes et Règles Métier Configurables

Résumé du Module

Priorité : P3 — Normale **Effort :** Small (1 semaine) **Impact :** Moyen

Ce module permet aux gestionnaires de configurer leurs propres règles d'alerte sans intervention technique. Il automatise la supervision et réduit la charge cognitive du gestionnaire.

9.1 Exigences Fonctionnelles

Exigence

EF-ALR01 — Règles configurables : Le gestionnaire doit pouvoir créer des règles d'alerte personnalisées basées sur des seuils métier : vitesse maximale, kilométrage journalier, niveau carburant minimum, durée de conduite maximale.

Exigence

EF-ALR02 — Canaux de notification : Chaque règle doit permettre de configurer les canaux de notification : push mobile, email, SMS, webhook.

Exigence

EF-ALR03 — Historique des alertes : Toutes les alertes déclenchées doivent être conservées avec leur contexte (valeur mesurée, seuil configuré, véhicule/conducteur concerné).

Exigence

EF-ALR04 — Acquiescement : Le gestionnaire doit pouvoir acquiescer une alerte pour indiquer qu'il en a pris connaissance et l'action entreprise.

9.2 Modèle de Données

Listing 9.1 : Tables du module Alertes et Règles

```

1  -- Regles d'alerte configurables par le gestionnaire
2  CREATE TABLE IF NOT EXISTS fleet.alert_rules (
3      id                UUID PRIMARY KEY,
4      fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
5      name              VARCHAR(255) NOT NULL,
6      description       TEXT,
7      rule_type         VARCHAR(50) NOT NULL CHECK (rule_type IN (
8          'SPEED_LIMIT',          -- Vitesse max
9          'DAILY_KM_LIMIT',       -- Km journaliers max
10         'FUEL_LEVEL_LOW',       -- Carburant bas
11         'DRIVING_HOURS_MAX',    -- Heures conduite max
12         'IDLE_TIME_MAX',        -- Temps moteur tourne
13         'GEOFENCE_VIOLATION',   -- Sortie de zone
14         'MAINTENANCE_OVERDUE',  -- Maintenance en
                                retard
15         'DOCUMENT_EXPIRING',    -- Document expirant
16         'INCIDENT_CRITICAL',    -- Incident critique
17         'BUDGET_THRESHOLD'      -- Seuil budget
18     )),
19     threshold_value    NUMERIC(12,2) NOT NULL,
20     threshold_unit     VARCHAR(20), -- km/h, km, %, h, jours
21     applies_to         VARCHAR(20) DEFAULT 'ALL'
22     CHECK (applies_to IN ('ALL', 'VEHICLE', 'DRIVER')),
23     entity_id          UUID, -- ID spécifique si applies_to != ALL
24     channels           VARCHAR(100) DEFAULT 'PUSH', -- PUSH, EMAIL, SMS
25     is_active          BOOLEAN DEFAULT true,
26     cooldown_minutes   INT DEFAULT 60, -- Delai entre deux alertes
                                identiques
27     created_at         TIMESTAMP DEFAULT now()
28 );
29
30 -- Historique des alertes declenchees
31 CREATE TABLE IF NOT EXISTS fleet.alert_history (
32     id                UUID PRIMARY KEY,
33     rule_id           UUID NOT NULL REFERENCES fleet.alert_rules(id)
34     ON DELETE CASCADE,
35     fleet_id          UUID NOT NULL REFERENCES fleet.fleets(id),
36     vehicle_id        UUID REFERENCES fleet.vehicles(id),
37     driver_id         UUID REFERENCES fleet.drivers(user_id),
38     triggered_at      TIMESTAMP NOT NULL DEFAULT now(),
39     measured_value    NUMERIC(12,2),
40     threshold_value    NUMERIC(12,2),
41     context_data       JSONB, -- Donnees contextuelles (position, etc.)

```

```
42     status          VARCHAR(20) DEFAULT 'OPEN'
43     CHECK (status IN ('OPEN', 'ACKNOWLEDGED', 'RESOLVED'))
44     acknowledged_by UUID,
45     acknowledged_at TIMESTAMP,
46     action_taken     TEXT
47 );
48
49 CREATE INDEX IF NOT EXISTS idx_alert_rules_fleet
50     ON fleet.alert_rules(fleet_id, is_active);
51 CREATE INDEX IF NOT EXISTS idx_alert_history_fleet
52     ON fleet.alert_history(fleet_id, triggered_at DESC);
53 CREATE INDEX IF NOT EXISTS idx_alert_history_open
54     ON fleet.alert_history(fleet_id, status)
55     WHERE status = 'OPEN';
```

9.3 Endpoints REST Proposés

Méthode URL		Rôle	HTTP
POST	/api/v1/alert-rules	MANAGER	201
GET	/api/v1/alert-rules	MANAGER	200
PUT	/api/v1/alert-rules/{id}	MANAGER	200
PATCH	/api/v1/alert-rules/{id}/toggle	MANAGER	200
DELETE	/api/v1/alert-rules/{id}	MANAGER	204
GET	/api/v1/alerts	MANAGER	200
GET	/api/v1/alerts/open	MANAGER	200
PATCH	/api/v1/alerts/{id}/acknowledge	MANAGER	200
GET	/api/v1/alerts/stats	MANAGER	200

TAB. 9.1 : Endpoints du module Alertes

Chapitre 10

Plan d'Implémentation Détaillé

10.1 Principes Directeurs

Avant de détailler le plan, voici les principes qui guident l'ordre d'implémentation :

1. **Valeur métier d'abord** : Les modules qui débloquent la commercialisation (Documents, KPIs) passent avant les modules d'enrichissement.
2. **Dépendances techniques** : Certains modules dépendent d'autres (Missions dépend de Planification, Scoring dépend des Trajets).
3. **Incrémentalité** : Chaque sprint livre un module fonctionnel et testable de bout en bout.
4. **Architecture hexagonale** : Chaque module suit les 6 phases définies dans le projet existant (Domaine → Ports → Services → Persistance → API → Intégration).

10.2 Roadmap par Sprints

10.2.1 Sprint 1 — Fondations Transversales (Semaine 1-2)

Sprint 1 — Prérequis Techniques

Objectif : Mettre en place les fondations techniques nécessaires à tous les modules.

Tâches :

- Ajouter le support de la **pagination** sur tous les endpoints existants (Pageable réactif avec PageRequest)
- Implémenter le **Soft Delete** : ajouter `deleted_at` aux tables critiques et filtrer dans les repositories
- Créer le **job planifié** de base (Spring `@Scheduled` ou Quartz) pour les tâches nocturnes
- Ajouter le support **multilingue** (FR/EN) pour les messages d'erreur
- Créer l'infrastructure d'**export CSV** réutilisable
- Migration Liquibase : `011-add-soft-delete.sql`

Livrable : Infrastructure technique prête pour tous les modules suivants.

10.2.2 Sprint 2 — Module Documents Légaux (Semaine 3-4)

Sprint 2 — Documents et Conformité

Objectif : Gestion complète des documents légaux avec alertes d'expiration.

Phase 1 — Domaine (2 jours) :

- Créer `VehicleDocument` et `DriverDocument` (classes domaine)
- Créer `DocumentException` avec codes `DOC_001` à `DOC_010`
- Créer `ManageDocumentUseCase` (port entrant)
- Créer `DocumentPersistencePort` (port sortant)

Phase 2 — Infrastructure (3 jours) :

- Migration Liquibase : `012-document-tables.sql`
- Entités R2DBC : `VehicleDocumentEntity`, `DriverDocumentEntity`
- Repositories et adapters de persistance
- `DocumentController` avec 9 endpoints

Phase 3 — Job planifié (1 jour) :

- `DocumentExpiryCheckJob` : scan nocturne, mise à jour des statuts, envoi notifications J-30/J-15/J-7

Livrable : Conformité documentaire complète avec alertes automatiques.

10.2.3 Sprint 3 — Module KPI et Rapports (Semaine 5-6)

Sprint 3 — Tableau de Bord KPI

Objectif : Calcul et exposition des KPIs métier de la flotte.

Phase 1 — Domaine (1 jour) :

- Créer `KpiSnapshot` (record domaine)
- Créer `KpiUseCase` avec méthodes de calcul et de requête
- Créer `KpiPersistencePort`

Phase 2 — Infrastructure (3 jours) :

- Migration Liquibase : `013-kpi-tables.sql`
- `KpiSnapshotEntity` et `KpiR2dbcRepository`
- `KpiCalculationService` : agrégation des données existantes (trajets, maintenances, incidents, carburant)
- `KpiController` avec 8 endpoints

Phase 3 — Job planifié (1 jour) :

- `KpiCalculationJob` : calcul quotidien, hebdomadaire et mensuel

Livrable : Dashboard KPI complet avec comparaisons temporelles.

10.2.4 Sprint 4 — Module Dépenses et Budget (Semaine 7-8)

Sprint 4 — Gestion Financière

Objectif : Centralisation des dépenses et pilotage budgétaire.

Tâches (10 jours) :

- Domaine : `Expense`, `Budget`, `ExpenseCategory` + ports
- Migration Liquibase : `014-expense-tables.sql`
- Service : `ExpenseService` avec workflow d'approbation (PENDING → APPROVED/REJECTED)
- Service : `BudgetService` avec calcul de consommation en temps réel
- Job : `BudgetAlertJob` — alerte à 80% de consommation
- Controllers : `ExpenseController`, `BudgetController`
- Intégration : lier les dépenses carburant et maintenance aux `expenses` automatiquement

Livrable : Maîtrise complète des coûts d'exploitation.

10.2.5 Sprint 5 — Module Planification (Semaine 9-11)

Sprint 5 — Planification et Ordonnancement

Objectif : Planification des services et affectations véhicule-conducteur.

Tâches (15 jours) :

- Domaine : `Schedule`, `Assignment` + ports + `PlanningException`
- Migration Liquibase : `015-planning-tables.sql`
- Service : `ScheduleService` avec publication/archivage
- Service : `AssignmentService` avec détection de conflits (requête de chevauchement temporel)
- Algorithme de suggestion d'affectation (disponibilité véhicule + conducteur)
- Controllers : `ScheduleController`, `AssignmentController`
- Intégration : lier les `assignments` aux `trips` existants

Livrable : Planification opérationnelle complète avec détection de conflits.

10.2.6 Sprint 6 — Module Scoring Conducteur (Semaine 12-14)

Sprint 6 — Scoring et Éco-conduite

Objectif : Analyse comportementale de la conduite et scoring.

Tâches (15 jours) :

- Domaine : `DrivingEvent`, `DriverScore` + ports
- Migration Liquibase : `016-scoring-tables.sql`
- Service : `DrivingEventService` — traitement des événements en temps réel
- Service : `ScoringService` — calcul du score par trajet et agrégation périodique
- Algorithme de scoring configurable (barème paramétrable)
- Enrichissement de l'endpoint `POST /trips/{id}/telemetry` pour détecter les événements
- Controllers : `ScoringController`
- Job : `ScoreAggregationJob` — calcul hebdomadaire et mensuel + classement

Livrable : Scoring conducteur complet avec classement et feedback.

10.2.7 Sprint 7 — Module Maintenance Préventive (Semaine 15-16)

Sprint 7 — Maintenance Préventive

Objectif : Planification automatique des maintenances préventives.

Tâches (10 jours) :

- Domaine : `MaintenancePlan`, `MaintenanceTask` + ports
- Migration Liquibase : `017-preventive-maintenance.sql`
- Service : `MaintenancePlanService` + `MaintenanceTaskService`
- Algorithme de calcul des prochaines échéances (km ou date)
- Job : `MaintenanceCheckJob` — vérification quotidienne des seuils, création automatique des tâches
- Intégration : mise à jour des `next_due_km` et `next_due_date` après chaque maintenance réelle
- Controllers : `MaintenancePlanController`, `MaintenanceTaskController`

Livrable : Zéro panne surprise grâce à la maintenance préventive.

10.2.8 Sprint 8 — Module Clients et Missions (Semaine 17-20)

Sprint 8 — Clients, Missions et Facturation

Objectif : Gestion commerciale complète avec facturation.

Tâches (20 jours) :

- Domaine : `Client`, `Mission`, `Invoice` + ports
- Migration Liquibase : `018-clients-missions.sql`
- Service : `ClientService`, `MissionService`, `InvoiceService`
- Génération de token de suivi public (UUID + expiration)
- Endpoint public `/track/{token}` sans authentification
- Génération PDF des factures (iText ou JasperReports)
- Intégration avec `assignments` (Sprint 5) pour l'affectation automatique
- Controllers : `ClientController`, `MissionController`, `InvoiceController`

Livrable : Plateforme commerciale complète avec suivi client en temps réel.

10.2.9 Sprint 9 — Module Alertes Configurables (Semaine 21)**Sprint 9 — Alertes et Règles Métier****Objectif :** Automatisation de la supervision via des règles configurables.**Tâches (5 jours) :**

- Domaine : `AlertRule`, `AlertHistory` + ports
- Migration Liquibase : `019-alert-rules.sql`
- Service : `AlertRuleService` + `AlertEvaluationService`
- Moteur d'évaluation des règles : intégration dans les services existants (`TripService`, `IncidentService`, etc.)
- Gestion du cooldown (éviter le spam d'alertes)
- Controllers : `AlertRuleController`, `AlertHistoryController`

Livrable : Supervision automatisée et personnalisable.**10.3 Tableau Récapitulatif du Plan**

Sprint Module		Semaines	Priorité	Effort	Dépendances
S1	Fondations	1-2	P1 — Critique	S	Aucune
S2	Documents	3-4	P1 — Critique	M	S1
S3	KPI	5-6	P1 — Critique	M	S1
S4	Dépenses	7-8	P2 Haute	M	S1, S3
S5	Planification	9-11	P1 — Critique	L	S1
S6	Scoring	12-14	P2 Haute	L	S5
S7	Maint. Préventive	15-16	P2 Haute	M	S5
S8	Clients/Missions	17-20	P3 Normale	L	S5, S4
S9	Alertes	21	P3 Normale	S	S2..S8

Sprint	Module	Semaines	Priorité	Effort	Dépendances
--------	--------	----------	----------	--------	-------------

TAB. 10.1 : Récapitulatif du plan d'implémentation (21 semaines)

10.4 Diagramme de Dépendances

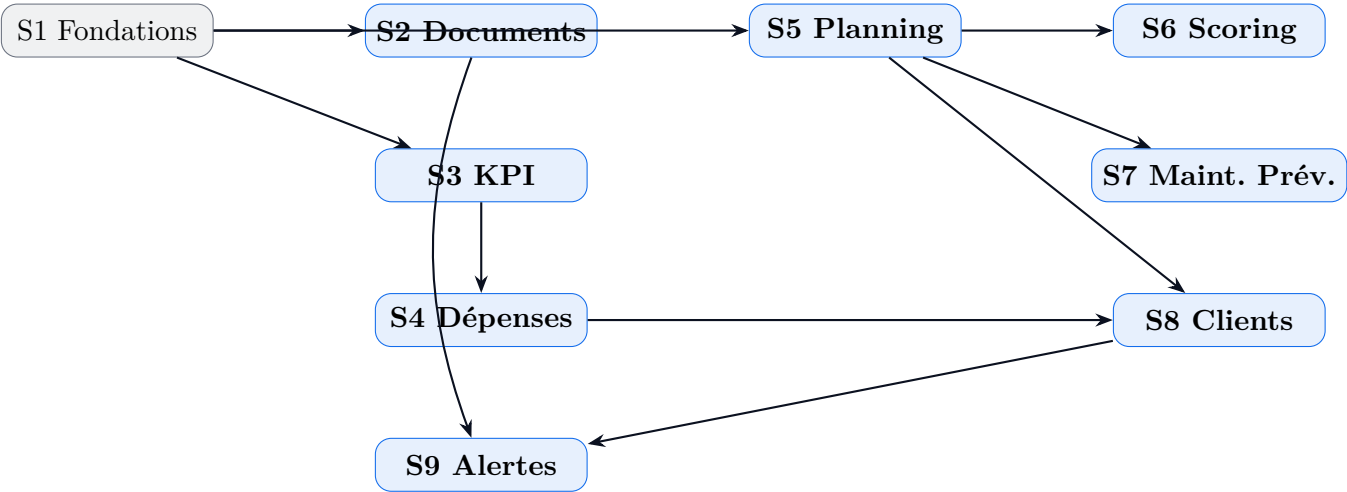


FIG. 10.1 : Graphe de dépendances entre les sprints

Chapitre 11

Architecture Technique des Nouveaux Modules

11.1 Patron d'Implémentation Standard

Chaque nouveau module suit rigoureusement le même patron hexagonal en 6 phases, identique à celui utilisé pour le module Opérations Terrain existant :

Phase	Couche	Livrables
1	Domaine	Entités/records, Value Objects, exceptions typées
2	Ports	Interfaces Use Case (in) + Persistence/Event ports (out)
3	Application	Services @Service implémentant les Use Cases
4	Persistance	Entités R2DBC, repositories, adapters, migration Liquibase
5	API REST	DTOs, controllers WebFlux, documentation Swagger
6	Intégration	Jobs planifiés, connexions inter-modules, KPIs

TAB. 11.1 : Patron d'implémentation hexagonal en 6 phases

11.2 Jobs Planifiés — Architecture

Les jobs planifiés constituent un composant transversal critique. Voici l'architecture proposée :

Listing 11.1 : Architecture d'un job planifié réactif

```
1 @Component
2 @RequiredArgsConstructor
3 @Slf4j
```

```

4 public class DocumentExpiryCheckJob {
5
6     private final DocumentPersistencePort documentPort;
7     private final SendNotificationPort notificationPort;
8
9     // Execution chaque nuit a 2h00
10    @Scheduled(cron = "0_0_2_*_*_*")
11    public void checkExpiringDocuments() {
12        log.info("Demarrage du job de verification des documents");
13
14        // Traitement reactif avec subscribe() pour ne pas bloquer
15        documentPort.findExpiringWithin(30)
16            .flatMap(doc -> updateStatusAndNotify(doc))
17            .doOnError(e -> log.error("Erreur job documents", e))
18            .subscribe(
19                null,
20                e -> log.error("Echec job documents", e),
21                () -> log.info("Job documents termine")
22            );
23    }
24
25    private Mono<Void> updateStatusAndNotify(DocumentDto doc) {
26        return documentPort.updateStatus(doc.id(), "EXPIRING_SOON")
27            .then(notificationPort.send(buildNotification(doc)));
28    }
29 }

```

Note Experte

Les jobs planifiés utilisent `@Scheduled` de Spring avec des expressions cron. Pour les environnements multi-instances (Kubernetes), il faut ajouter `ShedLock` ou `Spring Batch` pour éviter l'exécution parallèle sur plusieurs pods.

11.3 Nouvelles Migrations Liquibase

Le tableau suivant liste les 9 nouveaux changesets Liquibase à créer :

Fichier	Sprint	Contenu
011-add-soft-delete.sqlS1		Ajout <code>deleted_at</code> aux tables critiques
012-document-tables.sqlS2		<code>vehicle_documents</code> , <code>driver_documents</code> , <code>document_alerts</code>

Fichier	Sprint	Contenu
013-kpi-tables.sql	S3	kpi_snapshots
014-expense-tables.sql	S4	expense_categories, expenses, budgets
015-planning-tables.sql	S5	schedules, assignments
016-scoring-tables.sql	S6	driving_events, driver_scores
017-preventive-maintenance.sql	S7	maintenance_plans, maintenance_tasks
018-clients-missions.sql	S8	clients, missions, invoices
019-alert-rules.sql	S9	alert_rules, alert_history

TAB. 11.2 : Nouveaux changesets Liquibase

11.4 Nouveaux Tags Swagger

Pour maintenir la cohérence de la documentation API, les nouveaux modules s'intègrent dans la numérotation existante (01-14) :

Tag	Nom	Module
TAG_15	Documents Véhicules	Module 2 — Documents
TAG_16	Documents Conducteurs	Module 2 — Documents
TAG_17	KPIs & Rapports	Module 3 — KPI
TAG_18	Dépenses	Module 4 — Dépenses
TAG_19	Budgets	Module 4 — Dépenses
TAG_20	Plannings	Module 1 — Planification
TAG_21	Affectations	Module 1 — Planification
TAG_22	Scoring Conducteur	Module 5 — Scoring
TAG_23	Maintenance Préventive	Module 6 — Maintenance
TAG_24	Clients	Module 7 — Clients
TAG_25	Missions	Module 7 — Missions
TAG_26	Factures	Module 7 — Facturation
TAG_27	Règles d'Alerte	Module 8 — Alertes

TAB. 11.3 : Nouveaux tags Swagger

Chapitre 12

Analyse de Valeur et Priorisation

12.1 Matrice Valeur / Effort

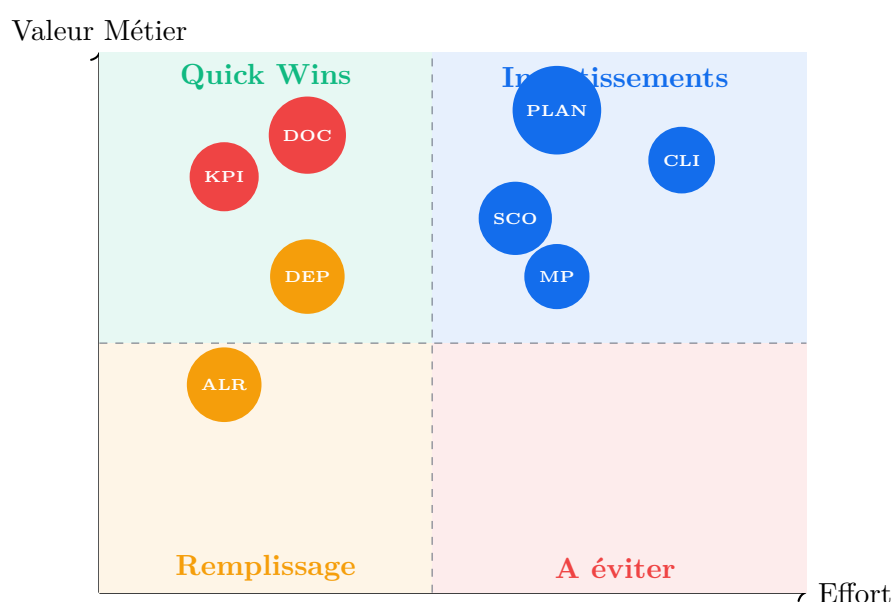


FIG. 12.1 : Matrice Valeur/Effort des 8 modules (DOC=Documents, KPI=KPIs, PLAN=Planification, CLI=Clients, SCO=Scoring, DEP=Dépenses, ALR=Alertes, MP=Maint. Préventive)

12.2 Retour sur Investissement Estimé

Module	ROI Estimé	Justification
Documents	Immédiat	Évite les amendes légales (assurance expirée = immobilisation)
KPI	1-3 mois	Réduction des coûts par optimisation basée sur les données
Scoring	3-6 mois	Réduction carburant 10-15%, réduction accidents 20-30%

Module	ROI Estimé	Justification
Maint. Préventive	3-6 mois	Réduction pannes 40%, coûts maintenance -25%
Dépenses	1-3 mois	Visibilité sur les coûts cachés, détection des fraudes
Planification	1-3 mois	Optimisation utilisation flotte, réduction temps mort
Clients/Missions	6-12 mois	Nouveau flux de revenus, facturation automatisée
Alertes	Immédiat	Réduction du temps de réaction aux incidents

TAB. 12.1 : Estimation du retour sur investissement par module

Conclusion

Ce cahier d'analyse et de conception a identifié **8 modules fonctionnels** à fort impact métier pour enrichir la plateforme FleetMan. Ces modules couvrent l'ensemble du cycle de vie de la gestion de flotte professionnelle :

- **Conformité légale** (Documents) — non-négociable pour opérer légalement
- **Pilotage de la performance** (KPI, Scoring) — différenciateur concurrentiel
- **Maîtrise des coûts** (Dépenses, Maintenance Préventive) — ROI direct et mesurable
- **Efficacité opérationnelle** (Planification, Alertes) — réduction de la charge cognitive
- **Monétisation** (Clients, Missions, Facturation) — nouveau modèle de revenus

Le plan d'implémentation proposé sur **21 semaines** (9 sprints) est conçu pour livrer de la valeur à chaque itération, en respectant scrupuleusement l'architecture hexagonale réactive existante. Les 18 nouvelles tables SQL, les 9 migrations Liquibase et les 80+ nouveaux endpoints REST s'intègrent naturellement dans le projet sans rupture architecturale.

Recommandation Finale

Ordre de priorité recommandé pour une mise en production rapide :

1. **Sprint 2 — Documents** : Conformité légale immédiate, risque zéro
2. **Sprint 3 — KPI** : Valeur perçue maximale pour les gestionnaires
3. **Sprint 5 — Planification** : Fonctionnalité la plus demandée sur le terrain
4. **Sprint 6 — Scoring** : Différenciateur fort face à la concurrence

Ces 4 sprints (10 semaines) suffisent à positionner FleetMan comme une solution premium sur le marché africain du transport.

Annexe A

Récapitulatif des Nouvelles Tables SQL

Table	Migration	Clés étrangères principales
fleet.schedules	015	fleet_id, manager_id
fleet.assignments	015	schedule_id, vehicle_id, driver_id
fleet.vehicle_documents	012	vehicle_id
fleet.driver_documents	012	driver_id
fleet.document_alerts	012	document_id
fleet.kpi_snapshots	013	fleet_id, entity_id
fleet.expense_categories	014	fleet_id
fleet.expenses	014	fleet_id, vehicle_id, driver_id, category_id
fleet.budgets	014	fleet_id, category_id
fleet.driver_scores	016	driver_id, fleet_id
fleet.driving_events	016	trip_id, driver_id, vehicle_id
fleet.maintenance_plans	017	fleet_id, vehicle_type_id
fleet.maintenance_tasks	017	plan_id, vehicle_id
fleet.clients	018	fleet_id
fleet.missions	018	fleet_id, client_id, assignment_id
fleet.invoices	018	fleet_id, client_id, mission_id
fleet.alert_rules	019	fleet_id
fleet.alert_history	019	rule_id, fleet_id, vehicle_id

TAB. A.1 : Récapitulatif des 18 nouvelles tables

Annexe B

Récapitulatif des Nouveaux Endpoints REST

Méthode	URL	Module	Rôle
POST	/api/v1/schedules	Planification	MANAGER
GET	/api/v1/assignments/conflicts	Planification	MANAGER
GET	/api/v1/assignments/driver/{id}/today	Planification	DRIVER
POST	/api/v1/vehicles/{id}/documents	Documents	MANAGER
GET	/api/v1/documents/expiring	Documents	MANAGER
GET	/api/v1/documents/compliance-report	Documents	MANAGER
GET	/api/v1/kpis/fleet/{id}	KPI	MANAGER
GET	/api/v1/kpis/fleet/{id}/compare	KPI	MANAGER
GET	/api/v1/kpis/fleet/{id}/top-drivers	KPI	MANAGER
POST	/api/v1/expenses	Dépenses	MANAGER/DRIVER
PATCH	/api/v1/expenses/{id}/approve	Dépenses	MANAGER
GET	/api/v1/budgets/consumption	Dépenses	MANAGER
GET	/api/v1/drivers/{id}/score	Scoring	MANAGER
GET	/api/v1/fleets/{id}/driver-ranking	Scoring	MANAGER
POST	/api/v1/trips/{id}/driving-events	Scoring	DRIVER
GET	/api/v1/maintenance-tasks/overdue	Maint. Prév.	MANAGER
GET	/api/v1/vehicles/{id}/maintenance-schedule	Maint. Prév.	MANAGER
POST	/api/v1/clients	Clients	MANAGER
POST	/api/v1/missions	Missions	MANAGER
GET	/api/v1/missions/{id}/available-vehicles	Missions	MANAGER
GET	/api/v1/track/{token}	Missions	PUBLIC
POST	/api/v1/invoices/generate/{missionId}	Facturation	MANAGER
GET	/api/v1/invoices/{id}/pdf	Facturation	MANAGER
POST	/api/v1/alert-rules	Alertes	MANAGER
GET	/api/v1/alerts/open	Alertes	MANAGER

Méthode	URL	Module	Rôle
PATCH	/api/v1/alerts/{id}/acknowledge	Alertes	MANAGER

TAB. B.1 : Sélection des nouveaux endpoints REST (liste non exhaustive)